PYTHON FOR DATA SCIENCE · LESSON 25 OF 29

# Performance and Bigger Data: Vectorisation, Memory, Parquet, Polars and DuckDB

Make Python data work fast and scale beyond memory. Measure with timeit, replace loops with vectorised code, shrink DataFrames with the right dtypes, store data as Parquet, use Polars lazy queries and DuckDB for out-of-core analytics, and know when to move to Spark and the cloud.

Python class 19 | ✦ pandas 3 · Polars 1.x/2.x · DuckDB · PyArrow | Intermediate | ⏱ 100 min

**By the end of this lesson you can**

- Measure speed and memory before optimising
- Replace row loops and apply with vectorised pandas and NumPy operations
- Reduce memory with categories, smaller numeric types and selected columns
- Store and read data efficiently with Parquet and explain columnar formats
- Write equivalent queries in pandas, Polars and DuckDB and choose between them

**Datasets used**

`superstore_sales.csv`

## Measure first

"Premature optimisation is the root of all evil" (Donald Knuth). Profile, find the slow part, fix that.

Python

```
import pandas as pd, numpy as np, time, timeit
s = pd.read_csv("superstore_sales.csv", parse_dates=["order_date"])
big = pd.concat([s] * 20, ignore_index=True)                # ~126k rows to make timings visible
print(big.shape)
def t(fn, n=3): return min(timeit.repeat(fn, number=1, repeat=n)) * 1000
loop = lambda: [r.sales * (1 - r.discount) for r in big.itertuples()]
apply_ = lambda: big.apply(lambda r: r["sales"] * (1 - r["discount"]), axis=1)
vec = lambda: big["sales"] * (1 - big["discount"])
print(f"itertuples loop {t(loop, 1):8.1f} ms\napply(axis=1)   {t(apply_, 1):8.1f} ms\nvectorised      {t(vec):8.2f} ms")
```

OUTPUT

```
(126460, 19)
itertuples loop   1070.5 ms
apply(axis=1)      659.5 ms
vectorised           0.65 ms
```

The vectorised version is more than a thousand times faster. `itertuples` and `apply(axis=1)` take about the same time because both are Python loops (`apply` with `axis=1` is a loop in disguise), and `iterrows` is slower still.

## Vectorising conditional logic

Python

```
def band_loop(d): return "high" if d > 0.2 else "mid" if d > 0 else "none"
f1 = lambda: big["discount"].map(band_loop)
f2 = lambda: np.select([big.discount > 0.2, big.discount > 0], ["high", "mid"], default="none")
f3 = lambda: pd.cut(big.discount, [-1, 0, 0.2, 1], labels=["none", "mid", "high"])
print(f"map(func) {t(f1):.1f} ms | np.select {t(f2):.1f} ms | pd.cut {t(f3):.1f} ms")
print((f1().values == f2()).all())
```

OUTPUT

```
map(func) 15.6 ms | np.select 1.0 ms | pd.cut 1.5 ms
True
```

| Slow pattern | Faster pattern |
| --- | --- |
| `for` / `iterrows` over rows | Column arithmetic, `np.where`, `np.select` |
| `apply(axis=1)` | Vectorised expression, or `groupby().transform` |
| Growing a DataFrame in a loop (`concat` each time) | Collect a list of dicts, build once |
| String ops in Python loops | `.str` methods |
| Repeated `df[df.x == k]` in a loop | One `groupby` |
| Reading a big CSV many times | Read once, save as Parquet |

## Memory

Python

```
def mb(df): return df.memory_usage(deep=True).sum() / 1e6
print(f"original: {mb(big):.1f} MB")
small = big.copy()
for c in ["region", "segment", "category", "sub_category", "ship_mode", "state", "city"]:
small[c] = small[c].astype("category")
small["quantity"] = pd.to_numeric(small["quantity"], downcast="integer")
small[["sales", "profit", "discount"]] = small[["sales", "profit",
"discount"]].astype("float32")
print(f"categories + downcast: {mb(small):.1f} MB")
print(small.dtypes.astype(str).value_counts().to_dict())
```

OUTPUT

```
original: 34.7 MB
categories + downcast: 18.3 MB
{'category': 7, 'str': 6, 'float32': 3, 'int64': 1, 'datetime64[us]': 1, 'int8': 1}
```

Categories store each label once plus small integer codes: ideal for columns with few distinct values. Also read only needed columns (`usecols=`) and stream with `chunksize=`.

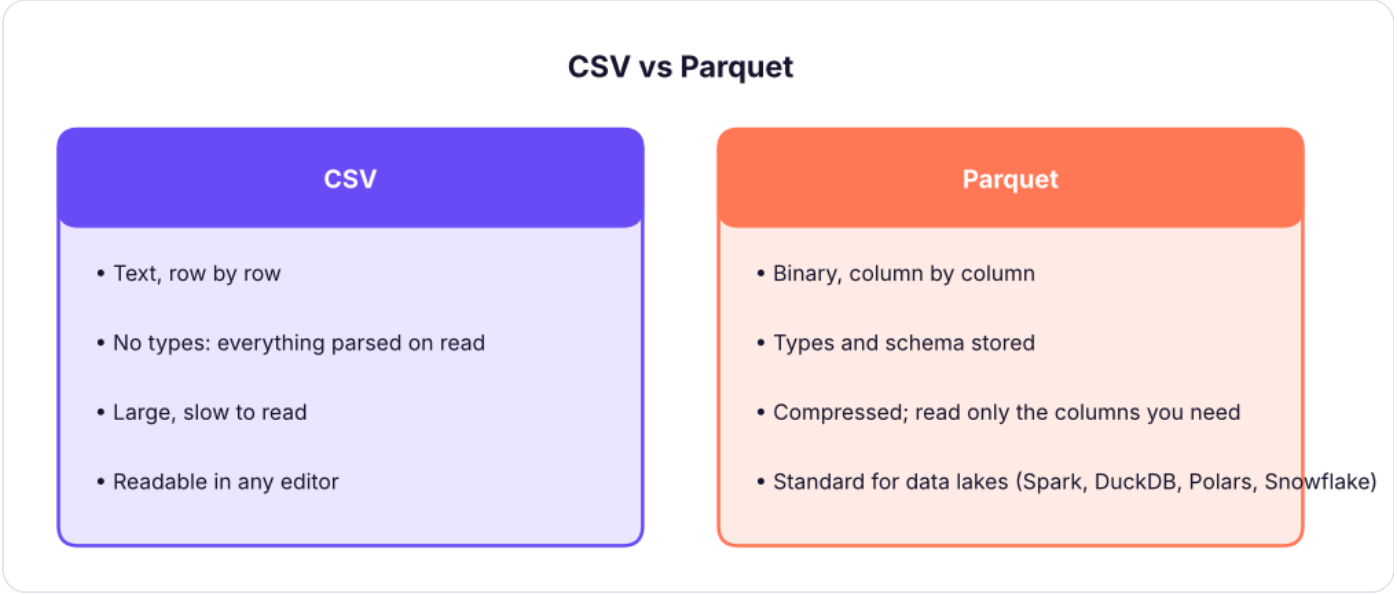
Python

```
total = 0.0
for chunk in pd.read_csv("superstore_sales.csv", usecols=["region", "sales"],
chunksize=2000):
    total += chunk.loc[chunk.region == "West", "sales"].sum()
print(f"West sales via chunks: {total:,.0f}")
```

OUTPUT

```
West sales via chunks: 51,958,520
```

## Parquet: the analytics file format




CSV is row-oriented text; Parquet stores each column together, typed and compressed.

Python

```python
import os
big.to_csv("big.csv", index=False); big.to_parquet("big.parquet", index=False)
print(f"CSV {os.path.getsize('big.csv')/1e6:.1f} MB | Parquet
{os.path.getsize('big.parquet')/1e6:.1f} MB")
print(f"read CSV {t(lambda: pd.read_csv('big.csv')):.0f} ms | read Parquet {t(lambda:
pd.read_parquet('big.parquet')):.0f} ms | two columns {t(lambda:
pd.read_parquet('big.parquet', columns=['region', 'sales'])):.0f} ms")
```

OUTPUT

```
CSV 21.7 MB | Parquet 1.9 MB
read CSV 249 ms | read Parquet 15 ms | two columns 5 ms
```

## Polars: fast DataFrames with lazy queries

**Polars** is a DataFrame library written in Rust. It uses all CPU cores, the Arrow memory format, and a **lazy** mode that optimises the whole query before running it.

Python

```python
import polars as pl
lazy = (pl.scan_parquet("big.parquet")                          # nothing is read yet
          .filter(pl.col("discount") ⩽ 0.1)
          .group_by("region", "category")
          .agg(sales=pl.col("sales").sum(), profit=pl.col("profit").sum(), lines=pl.len())
          .with_columns(margin=(100 * pl.col("profit") / pl.col("sales")).round(1),
sales_m=(pl.col("sales") / 1e6).round(2))
          .drop("profit")
          .sort("sales", descending=True))
print(lazy.head(4).collect().drop("sales"))
pandas_way = lambda: (big[big.discount ⩽ 0.1].groupby(["region", "category"]).agg(sales=
("sales", "sum"), profit=("profit", "sum")))
print(f"pandas {t(pandas_way):.1f} ms | polars lazy {t(lambda: lazy.collect()):.1f} ms |
polars {pl.__version__}")
```

OUTPUT

```
shape: (4, 5)
```

| region | category | lines | margin | sales_m |
|---|---|---|---|---|
| str | str | u32 | f64 | f64 |
| South | Technology | 7220 | 15.9 | 642.79 |
| West | Technology | 7080 | 16.1 | 604.14 |
| North | Technology | 6720 | 16.6 | 598.63 |
| East | Technology | 6320 | 17.5 | 531.33 |

```
pandas 22.0 ms | polars lazy 3.9 ms | polars 2.1.0
```

| pandas | Polars |
|---|---|
| `df[df.x > 1]` | `df.filter(pl.col("x") > 1)` |
| `df.assign(y=df.x * 2)` | `df.with_columns(y=pl.col("x") * 2)` |
| `df.groupby("g").agg(s=("x", "sum"))` | `df.group_by("g").agg(s=pl.col("x").sum())` |
| `df.sort_values("x")` | `df.sort("x")` |
| `pd.read_csv` | `pl.read_csv` / lazy `pl.scan_csv` |
| `df.to_pandas()` ↔ `pl.from_pandas(df)` | Convert when a library needs pandas |

## DuckDB on files larger than memory

Python

```
import duckdb
q = """
SELECT region, category, ROUND(SUM(sales)/1e6, 2) AS sales_m,
ROUND(100*SUM(profit)/SUM(sales), 1) AS margin
FROM 'big.parquet' WHERE discount ⩽ 0.1 GROUP BY ALL ORDER BY sales_m DESC LIMIT 4
"""
print(duckdb.sql(q).df())
print(f"duckdb {t(lambda: duckdb.sql(q).fetchall()):.1f} ms")
for f in ("big.csv", "big.parquet"): os.remove(f)
```

OUTPUT

```
  region    category  sales_m  margin
0  South  Technology   642.79    15.9
1   West  Technology   604.14    16.1
2  North  Technology   598.63    16.6
3   East  Technology   531.33    17.5
duckdb 7.0 ms
```

DuckDB and Polars stream data in batches, so they can process files bigger than RAM on a laptop. Folders of Parquet files ( `'data/*.parquet'` ) work the same way.

## Going further

| Need | Tool |
|---|---|
| Use all CPU cores for custom Python | `concurrent.futures` , `joblib.Parallel` , multiprocessing |
| Speed up numeric Python loops | Numba ( `@njit` ), Cython |
| Data bigger than one machine | Apache Spark (PySpark), Databricks, Dask, Ray |
| Warehouse-scale SQL | BigQuery, Snowflake, Redshift, Databricks SQL |
| GPU DataFrames | RAPIDS cuDF ( `cudf.pandas` ) |

Rule of thumb: up to a few GB → pandas with good dtypes; up to ~100 GB on one machine → Polars or DuckDB with Parquet; beyond that or many users → Spark or a cloud warehouse.

# Summary

**KEY TAKEAWAYS**

- Measure with `timeit` / `%timeit` before optimising; fix the bottleneck.
- Vectorise: column arithmetic, `np.where` / `np.select`, `pd.cut`, `.str`, `groupby`; avoid `iterrows` and `apply(axis=1)`.
- Save memory with categories, downcasting, `usecols` and `chunksize`.
- Parquet is columnar, typed and compressed: faster and smaller than CSV.
- Polars (lazy, multi-core) and DuckDB (SQL, out-of-core) handle data beyond pandas; Spark and warehouses go beyond one machine.

## Quick quiz

6 questions · tap an answer to check it

**1. Which is usually fastest for row-wise arithmetic?**

A iterrows

B apply(axis=1)

C A vectorised column expression

D A while loop

**Answer: C.** Vectorised code runs in compiled loops.

**2. Why convert a text column with 4 distinct values to `category`?**

- **A** It stores each label once with small integer codes, saving memory and speeding groupby
- **B** It sorts the data
- **C** It is required for charts
- **D** It removes missing values

**Answer: A.** Ideal for low-cardinality columns.

**3. What is the main advantage of Parquet over CSV?**

- **A** Human-readable
- **B** Columnar, typed and compressed, so it is smaller and faster to read
- **C** Supported only by Excel
- **D** No schema

**Answer: B.** You can read just the columns you need.

**4. What does Polars' `scan_parquet(...)` do?**

- **A** Reads the whole file immediately
- **B** Creates a lazy query that is optimised and executed on `.collect()`
- **C** Writes a file
- **D** Converts to CSV

**Answer: B.** Lazy evaluation enables predicate and projection push-down.

**5.** **Which tool queries Parquet files with SQL without a server?**

A MySQL

B DuckDB

C Excel

D requests

**Answer: B.** It also queries pandas DataFrames.

**6.** **When should you consider Spark or a cloud warehouse?**

A For 10,000 rows

B When data or users exceed what one machine can handle

C Always

D Never

**Answer: B.** Start simple; scale when needed.

EXERCISE **Practice**

(1) Time `iterrows`, `apply(axis=1)` and a vectorised version of `profit / sales` on `big`.
(2) Reduce the memory of `superstore_sales.csv` by at least 50% and report the percentage.
(3) Rewrite a pandas groupby from Lesson 18 in Polars and in DuckDB SQL and check that the numbers match.

**Show solution (worked answer)**

Python

```
print(t(lambda: [r["profit"] / r["sales"] for _, r in big.head(20000).iterrows()], 1))
print(t(lambda: big.profit / big.sales))
before = mb(s); s2 = s.copy()
for c in s2.select_dtypes("object").columns.union(s2.select_dtypes("string").columns):
    if s2[c].nunique() < 100: s2[c] = s2[c].astype("category")
print(f"{100 * (1 - mb(s2) / before):.0f}% smaller")
pl_res =
pl.from_pandas(s).group_by("region").agg(pl.col("sales").sum()).sort("region")
dd_res = duckdb.sql("SELECT region, SUM(sales) sales FROM s GROUP BY region ORDER BY
region").df()
```

INTERVIEW CORNER **Questions you may be asked**

**Q: Your pandas script is slow. What do you do?** Profile to find the slow step; vectorise loops and `apply`; reduce data early (filter, `usecols`); fix dtypes; avoid repeated work; save intermediate results as Parquet; consider Polars/DuckDB or pushing work to SQL.

**Q: Why is Parquet popular in data lakes?** Columnar storage enables compression and reading only needed columns; it stores the schema; and every major engine reads it.

**Q: pandas vs Polars?** pandas has the biggest ecosystem; Polars is faster and more memory-efficient thanks to Rust, multi-threading and lazy optimisation. Many teams use both and convert where needed.

## Practise more online

Free, trusted places to test yourself on this topic. Links open the original site in a new tab.